Talking to GemStone/S from Rust

A complete article-style guide to gemstone-rs

Generated from the Markdown sources in gemstone-rs/docs.

Contents

Talking to GemStone/S from Rust - A Complete Guide to gemstone-rs

GemStone/S stores live objects. Rust gives you fast, explicit systems code. `gemstone-rs` connects the two without putting Python in the process.

What is GemStone/S?

GemStone/S 64 is an object database and application server for Smalltalk systems. It stores objects directly, supports many concurrent sessions, and lets clients send Smalltalk messages to live objects. Instead of translating your application into rows and joins, you talk to objects in the stone.

Rust is a good fit for a direct GemStone client because Rust applications often need predictable resource management, explicit error handling, and clean deployment. `gemstone-rs` is the Rust bridge for that job.

The Architecture

```
Rust application
      |
      v
gemstone-rs             safe Rust API
      |
      v
gemstone-gci            dynamic libgcirpc loader and raw ABI calls
      |
      v
GCI C library           ships with GemStone/S
      |
      v
GemStone stone
```

The split matters:

Layer	Responsibility
`gemstone-gci`	Load `libgcirpc`, expose raw GCI calls, keep unsafe ABI code isolated.
`gemstone-rs`	Provide `Config`, `Session`, `Oop`, `Value`, transactions, browser API, and codegen.
`gemstone-rs-cli`	Provide `gemstone-rs eval`, browse, inspect, and codegen commands.
`gemstone-rs-explorer`	Provide a local-only HTTP explorer for browsing and codegen workflows.
`gemstone-rs Workbench`	Add VS Code sidebar browsing and codegen actions over the CLI.

This gives Rust users a native path to GemStone/S, while Python remains only one possible consumer through `gemstone-py`.

Install

For Rust applications:

```
cargo add gemstone-rs
```

For command-line tools:

```
cargo install gemstone-rs-cli
cargo install gemstone-rs-explorer
```

For VS Code:

```
https://marketplace.visualstudio.com/items?itemName=unicompute.gemstone-rs-workbench
```

Runtime configuration comes from the same environment variables used by the CLI, explorer, and workbench:

```
export GS_LIB=/opt/gemstone/product/lib
export GS_STONE=gs64stone
export GS_STONE_NAME=gs64stone
export GS_USERNAME=DataCurator
export GS_PASSWORD=swordfish
```

`GS_STONE` is canonical. `GS_STONE_NAME` is accepted as an alias when `GS_STONE` is absent. Set `GS_LIB_PATH` when you want to point directly at a specific `libgcirpc` file.

First Login

```
use gemstone_rs::{Config, Session, Value};

fn main() -> gemstone_rs::Result<()> {
    let mut session = Session::login(Config::from_env()?)?;

    let value = session.eval("3 + 4")?;
    assert_eq!(value, Value::SmallInt(7));
```

```
    session.logout()?;
    Ok(())
}
```

Run the same check from the CLI:

```
gemstone-rs doctor
gemstone-rs doctor --live
gemstone-rs doctor --json
gemstone-rs eval "3 + 4"
```

Expected output:

```
7
```

OOPs and Values

GemStone objects are identified by OOPs. `gemstone-rs` keeps that explicit:

```
use gemstone_rs::{Config, Session, Value};

let mut session = Session::login(Config::from_env()?)?;

let seven = session.value_to_oop(&Value::SmallInt(7))?;
let printed = session.perform_oop(seven, "printString", &[])?;
println!("{}", session.fetch_string(printed)?);
```

`Value` covers the simple automatic conversions:

GemStone result	Rust value
`nil`	`Value::Nil`
`true` / `false`	`Value::Bool`
`SmallInteger`	`Value::SmallInt`
`Character`	`Value::Char`
fetched string values	`Value::String`

GemStone result	Rust value
other live objects	`Value::Oop`

For long-lived raw OOPs, retain them in the GemStone export set:

```
let text = session.new_string("retained")?;
let handle = session.retain_oop(text)?;
println!("{}", handle.oop().raw());
handle.release()?;
```

The handle releases on drop if you do not call `release()`.

Transactions

GemStone sessions are transactional. `gemstone-rs` gives you manual primitives and a small transaction wrapper:

```
session.transaction(|session| {
    let value = session.new_string("committed from Rust")?;
    session.global_put("GemStoneRsExample", value)
})?;
```

If the closure returns `Ok`, the transaction commits. If it returns `Err`, the session aborts.

Manual control is also available:

```
let needs_commit = session.needs_commit()?;
let in_transaction = session.in_transaction()?;
session.commit()?;
session.abort()?;
```

Browser API

The reusable browser API is the foundation for the CLI, explorer, and VS Code sidebar:

```
use gemstone_rs::browser::{Browser, ALL_PROTOCOLS};

let mut browser = Browser::new(&mut session);
```

```
let dictionaries = browser.dictionaries()?;
let classes = browser.classes("UserGlobals")?;
let protocols = browser.protocols("Object", false, "")?;
let methods = browser.methods("Object", ALL_PROTOCOLS, false, "")?;
let source = browser.source("Object", "printString", false, "")?;
```

CLI equivalents:

```
gemstone-rs browse dictionaries
gemstone-rs browse classes UserGlobals
gemstone-rs browse protocols Object
gemstone-rs browse methods Object "-- all --"
gemstone-rs browse source Object printString
```

Codegen

Codegen turns a small, reviewable config into checked-in Rust wrappers.

```
output = examples/codegen/generated/gemstone_wrappers.rs
class = Object
method = Object>>printString | return=String | doc=Return the receiver printString.
method = Object>>class
```

Commands:

```
gemstone-rs codegen preview examples/codegen/gemstone-rs.codegen
gemstone-rs codegen diff examples/codegen/gemstone-rs.codegen
gemstone-rs codegen check examples/codegen/gemstone-rs.codegen
gemstone-rs codegen generate examples/codegen/gemstone-rs.codegen
```

Generate a starter config from a live stone:

```
gemstone-rs codegen discover gemstone-rs.codegen Object
```

Generated wrappers keep selector spelling in one place:

```
pub fn print_string(&mut self) -> Result<String> {
    let value = self.session.perform(self.oop, "printString", &[])?;
    match value {
        Value::String(value) => Ok(value),
        Value::Oop(oop) => self.session.fetch_string(oop),
        other => Err(Error::UnexpectedType {
            expected: "String",
            actual: format!("{other:?}"),
```

```
        }),
    }
}
```

The output is normal Rust code. Check it in so reviews and editor indexing stay predictable.

Object Mapping with BridgeRoot

Direct OOP access is important, but application code often wants normal Rust structs at the boundary. `gemstone-rs` now has a BridgeRoot mapping layer for that.

The default BridgeRoot is a GemStone `Dictionary` stored in `UserGlobals` under `#GemStoneRsBridgeRoot`. Rust can put typed payloads there, commit them, and read them back without hiding the lower-level OOP API.

The smallest version stores a plain bridge value:

```
use gemstone_rs::{BridgeValue, Config, Session};

let mut session = Session::login(Config::from_env()?)?;
let mut bridge_root = session.bridge_root()?;

let payload = BridgeValue::dictionary([
    ("name".to_string(), BridgeValue::from("Tariq")),
    ("amount".to_string(), BridgeValue::from(100_i64)),
    ("currency".to_string(), BridgeValue::from("GBP")),
]);

bridge_root.put("MyTestDict", payload)?;
bridge_root.commit()?;
```

For typed Rust code, derive `BridgeMapped`:

```
use gemstone_rs::{BridgeMapped, Config, Session};

#[derive(Clone, Debug, Eq, PartialEq, BridgeMapped)]
struct CustomerDraft {
    #[bridge(key_type = "Symbol")]
    name: String,
}

#[derive(Clone, Debug, Eq, PartialEq, BridgeMapped)]
struct BookingDraft {
    #[bridge(key = "amount", key_type = "Symbol")]
    amount: i64,
    customer: CustomerDraft,
    tags: Vec<String>,
```

```
}

let mut session = Session::login(Config::from_env()?)?;
let mut bridge_root = session.bridge_root()?;

let draft = BookingDraft {
    amount: 100,
    customer: CustomerDraft {
        name: "Tariq".to_string(),
    },
    tags: vec!["priority".to_string(), "demo".to_string()],
};

bridge_root.transaction(|root| {
    root.put_mapped("DerivedBookingDraft", &draft)?;
    let loaded: BookingDraft = root.get_mapped("DerivedBookingDraft")?;
    assert_eq!(loaded, draft);
    Ok(())
})?;
```

That example shows the important mapping choices:

Feature	Why it matters
`#[derive(BridgeMapped)]`	Normal Rust structs can become BridgeRoot payloads.
`#[bridge(key = "...")]`	Rust field names do not have to match GemStone dictionary keys.
`key_type = "Symbol"`	Symbol-keyed dictionaries are explicit instead of accidental.
nested structs	nested dictionaries can read back into nested Rust structs.
`Vec<T>`	GemStone arrays can read back into Rust vectors.
`transaction`	BridgeRoot writes can commit on success and abort on error.

Codegen can create the same mapping structs from config:

```
mapped = BookingDraft | doc=A typed Rust payload stored under BridgeRoot.
field = BookingDraft.name | type=String | key=name
field = BookingDraft.amount | type=SmallInt | key=amount | key_type=Symbol
field = BookingDraft.tags | type=Vec<String> | key=tags
```

Generate a mapping proposal from a live stone:

```
gemstone-rs codegen discover-mapping gemstone-rs.codegen BookingDraft Object
```

Then preview, diff, check, and generate as usual:

```
gemstone-rs codegen preview gemstone-rs.codegen
gemstone-rs codegen diff gemstone-rs.codegen
gemstone-rs codegen check gemstone-rs.codegen
gemstone-rs codegen generate gemstone-rs.codegen
```

The local explorer and VS Code extension expose this workflow too:

```
curl -s http://127.0.0.1:8787/api/bridge/root
curl -s http://127.0.0.1:8787/api/bridge/keys
curl -s 'http://127.0.0.1:8787/api/bridge/put?key=WorkbenchDraft&value=hello'
curl -s 'http://127.0.0.1:8787/api/bridge/remove?key=WorkbenchDraft'
curl -s 'http://127.0.0.1:8787/api/bridge/mapping-config?mapped=BookingDraft'
```

The explorer stays read-only unless started with `--allow-write`, so these BridgeRoot write endpoints are useful smoke tests without becoming accidental public write APIs.

In VS Code, use:

- `GemStone RS: Generate Mapping Config`
- `GemStone RS: Preview BridgeRoot`
- `GemStone RS: List BridgeRoot Keys`
- `GemStone RS: Put BridgeRoot String`
- `GemStone RS: Remove BridgeRoot Key`
- `GemStone RS: Run Generated Mapping Example`

This is still explicit object mapping, not transparent persistence. That is the right tradeoff for the first Rust layer: Rust callers can review every field, choose string or symbol keys, keep OOPs visible, and still avoid repetitive dictionary boilerplate.

Local Explorer

Run:

```
gemstone-rs-explorer --port 8787
```

Open:

```
http://127.0.0.1:8787/
```

Useful endpoints:

```
/api/status
/api/browse/dictionaries
/api/browse/classes?dictionary=UserGlobals
/api/browse/methods?class=Object&protocol=--%20all%20--
/api/browse/source?class=Object&selector=printString
/api/codegen/preview?config=examples/codegen/gemstone-rs.codegen
/api/codegen/diff?config=examples/codegen/gemstone-rs.codegen
/api/codegen/profiles?profile_file=examples/codegen/gemstone-rs.codegen-profiles.json
```

The explorer home page is now a usable UI rather than just a list of links. It lets you browse dictionaries/classes/protocols/methods/source, inspect BridgeRoot, list keys, run codegen checks, and exercise write-gated BridgeRoot edits from a browser.

The codegen workflow now has project profiles. A profile captures the codegen root, config path, mapped Rust type, and GemStone class. Local profiles stay in browser storage, while project profiles can live in a checked-in file such as:

```
examples/codegen/gemstone-rs.codegen-profiles.json
```

That means a team can keep named workflows like `default`, `object-wrapper`, and `bridge-mapping` beside the codegen config. The explorer can import/export profile JSON, show which imported profiles are new, replaced, or unchanged, and save project profile files only when started with `--allow-write`.

The same schema is available from the CLI, so profile files can be checked in CI:

```
gemstone-rs profile sample
gemstone-rs profile init gemstone-rs.codegen-profiles.json
gemstone-rs profile validate gemstone-rs.codegen-profiles.json
gemstone-rs profile validate --json gemstone-rs.codegen-profiles.json
gemstone-rs profile list gemstone-rs.codegen-profiles.json
gemstone-rs profile show default gemstone-rs.codegen-profiles.json
gemstone-rs profile resolve default gemstone-rs.codegen-profiles.json
gemstone-rs codegen check-profile default gemstone-rs.codegen-profiles.json
```

Write endpoints are deliberately constrained. Relative `config=` and `profile_file=` writes stay under the configured codegen root, `..` traversal in `config=`, `profile_file=`, or `root=` is rejected after URL decoding, and absolute write paths require an explicit `--allow-absolute-write-paths` opt-in. Project profile files are schema-validated before writing, including required unique names and string-valued `config`, `root`, `mapped`, and `className` fields.

The explorer binds to loopback by default, starts read-only, and requires explicit flags for eval and write operations.

VS Code Workbench

The VS Code extension adds a GemStone RS sidebar:

- dictionaries
- classes
- protocols
- methods
- method source
- BridgeRoot key listing
- BridgeRoot put/remove smoke actions
- codegen preview/diff/check/generate
- profile-driven codegen preview/diff/check/generate
- explorer launch
- embedded explorer webview

For a source checkout:

```
{
  "gemstoneRs.checkoutPath": "/path/to/gemstone-rs",
  "gemstoneRs.useCargo": true,
  "gemstoneRs.codegenConfig": "examples/codegen/gemstone-rs.codegen"
}
```

The extension stays thin. The Rust CLI remains the contract, which keeps the tooling testable outside VS Code.

For web services, keep GemStone calls on a blocking worker and treat `Session` as thread-local. The repository includes an Axum service sketch that shows the recommended shape without adding Axum as a core dependency.

The newest workbench setup check uses the same CLI `gemstone-rs doctor` report, and the CLI also has `doctor --json`, so terminal diagnostics, VS Code diagnostics, and release automation can all agree.

What to Run First

From the repository root:

```
cargo run -p gemstone-rs --example quickstart
cargo run -p gemstone-rs --example browser
cargo run -p gemstone-rs --example live_smoke_cookbook
cargo run -p gemstone-rs --example oop_values
```

```
cargo run -p gemstone-rs --example transactions
cargo run -p gemstone-rs --example codegen_workflow
```

For CI and release confidence:

```
make verify
DRY_RUN=1 scripts/release_all.sh 0.2.0
scripts/publish_verify.sh 0.2.0
```

Where gemstone-rs Fits

Use `gemstone-rs` when you want Rust services, CLIs, workers, explorers, or developer tools to talk to GemStone/S directly. Use `gemstone-py` when your application is Python-native. The shared design direction is clear: keep the low-level bridge small, make the session model explicit, and build higher-level tooling on top of the same stable API.

The long-term native direction is for `gemstone-py-native` to become a thin PyO3 wrapper over the Rust core. That would make Rust the shared GCI bridge and Python the ergonomic Python API on top.

> This PDF was generated locally from `docs/`. If you edit the Markdown sources, rerun `python docs/build_pdf_docs.py`.